

```
// ❌ آرایه به جای فیلدهای واقعی
public class User
{
    public string[] Data = new string[5];

    // ثابت‌های رشته‌ای ایندکس می‌کنی
    public const string NAME = "0";
    public const string EMAIL = "1";
    public const string PHONE = "2";
}

// استفاده فاجعه:
var user = new User();
user.Data[int.Parse(User.NAME)] = "Ali"; // واقعا؟ 🤖
```

🎯 خلاصه

Primitive Obsession یعنی وسواس استفاده از `decimal` , `bool` , `int` , `string` و آرایه‌ها، در حالی که می‌تونستی کلاس‌های کوچک و معنادار بسازی که هم اعتبارسنجی دارن، هم رفتار، هم خوانایی بهتری دارن.

حالا بفرما لینک اولین درمان

Replace Data Value with Object 

<https://refactoring.guru/replace-data-value-with-object>

Link reading is unavailable in Expert Mode. Please use Instant Mode.

📖 Replace Data Value with Object

تعریف ساده

یه فیلد ساده (مثل `string` , `int` , `decimal`) که منطق و اعتبارسنجی خاص خودش رو می‌طلبه، تبدیل کن به یه کلاس جداگانه.

🧠 کی به این تکنیک نیاز داریم؟

نشانه	توضیح
اعتبارسنجی تکراری	هرجا از <code>Phone</code> استفاده می‌کنی، باید چک کنی فرمتش درسته
منطق همراه داده	مثلاً فرمت نمایش، مقایسه، محاسبه
تکامل تدریجی	اول یه <code>string</code> ساده بود، حالا کلی منطق دورش جمع شده

مثال ساده

قبل از ریفاکتور:

```
csharp

public class Order
{
    public string Phone { get; set; }
}

// همه جا اعتبارسنجی پراکنده
if (order.Phone.StartsWith("09") && order.Phone.Length == 11)
{
    // ارسال پیامک
}
```

بعد از Replace Data Value with Object:

```
csharp

public class PhoneNumber
{
    private readonly string _value;

    public PhoneNumber(string value)
    {
        if (!value.StartsWith("09") || value.Length != 11)
            throw new ArgumentException("شماره موبایل نامعتبر");
        _value = value;
    }

    public string Value => _value;

    public override string ToString() => _value;
}

public class Order
{
    public PhoneNumber Phone { get; set; } // دیگه فقط معتبر می‌تونه باشه
}
```

مزایا

مزیت	توضیح
اعتبارسنجی یه جا	خودش فرمت رو چک می‌کنه PhoneNumber
خوانایی	از string معنادارتره PhoneNumber
جلوگیری از باگ	نمی‌تونی شماره اشتباه وارد کنی
منطق همراه داده	فرمت نمایش، مقایسه و... توی خود کلاسه

تمرین

ریفکتور کن **Replace Data Value with Object** این کد رو با

```
csharp

public class Employee
{
    public string Email { get; set; }
    public decimal Salary { get; set; }
}

public class PayrollService
{
    public void SendSalarySlip(Employee emp)
    {
        // اعتبارسنجی پراکنده ایمیل
        if (string.IsNullOrEmpty(emp.Email) || !emp.Email.Contains("@"))
            throw new Exception("ایمیل نامعتبر است");

        // اعتبارسنجی پراکنده حقوق
        if (emp.Salary < 0)
            throw new Exception("حقوق نمیتواند منفی باشد");

        // فرمت نمایش حقوق
        string formattedSalary = string.Format("{0:N0} تومان", emp.Salary);

        Console.WriteLine($"ارسال شد {formattedSalary} : {emp.Email} فیش حقوقی به");
    }
}
```

راهنمایی:

- یه کلاس Email بساز که توی سازنده @ رو چک کنه
- یه کلاس Money بساز که منفی بودن رو چک کنه + فرمت نمایش
- رو با این کلاس‌های جدید بازنویسی کن Employee
- رو ساده‌سازی کن PayrollService

 بفرست تا بررسی کنیم